CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)

Assistant
Teaching Professor
Lisa Yan

Caches IV: Set Associative; Cache Optimizations

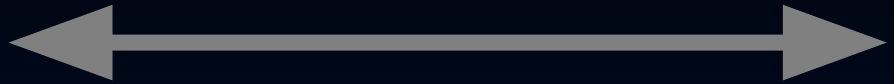
Cache design policies (1/2)

Placement policy: Where can a line be placed in the cache? How is a line found, e.g., how to detect cache hit?
Replacement policy: What happens on a cache miss?
Write policy: What happens on a write?
Hardware tradeoffs?

Cache design policies (2/2)

	Fully Associative Cache	Direct-Mapped Cache
Placement policy: Where can a line be placed in the cache? How is a line found, e.g., how to detect cache hit?	Data at any memory address can be associated with any cache line	Each memory address is associated with exactly one possible line in the cache.
Replacement policy: What happens on a cache miss?	Must decide. LRU or FIFO or Random	Know exactly which line to replace–the one at our target index
Write policy: What happens on a write?	Write-back (dirty bit) or write-through	Write-back (dirty bit) or write-through
Hardware tradeoffs?	Expensive. Check all tags.	Simple. Check one tag.

Cache Design: Placement Policies, So Far



Fully Associative Cache

Put a new line anywhere
(Caches II)

Set-Associative Cache

Something in-between

(Caches IV, **today**)

Direct Mapped Cache

Put a new line in one specific place
(Caches III)

Agenda

Set-Associative Caches

- Set-Associative Caches
- Cache Associativity
- Types of Misses
- Cache Optimizations

Set-Associative Cache

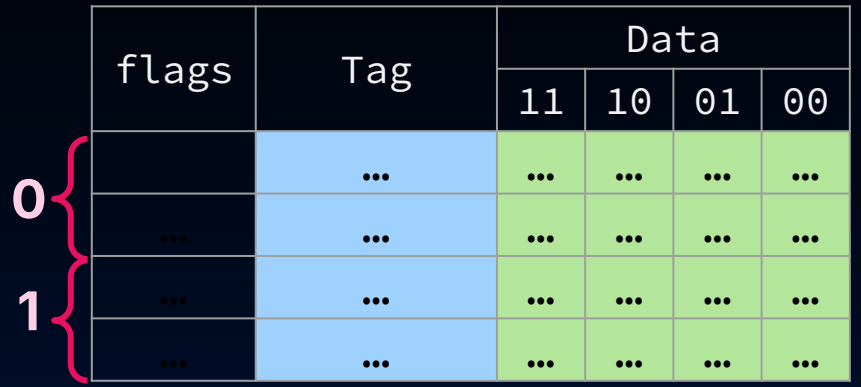
- **Placement policy**: The data can only be stored at one index, but the index has multiple entries.
  - To check for existence in the cache, we check all cache lines **associated with the same index**.
- **N-Way Set Associative**: sets of N cache lines are assigned a unique index

	flags	Tag	Data			
			11	10	01	00
0		…	…	…	…	…
	…	…	…	…	…	…
1	…	…	…	…	…	…
	…	…	…	…	…	…

2-Way Set Associative, cache size: 16B, line size 4B

Set-Associative Cache

- 2-Way Set Associative (right):
  - 4 lines
  - Each index is associated with a set of N=2 lines
    - Index: 0, 1
- **Associativity** of a cache: The number of slots (i.e., **ways**) assigned to each indexed set.
  - Block is first mapped to a set
  - Then placed in a way of that set



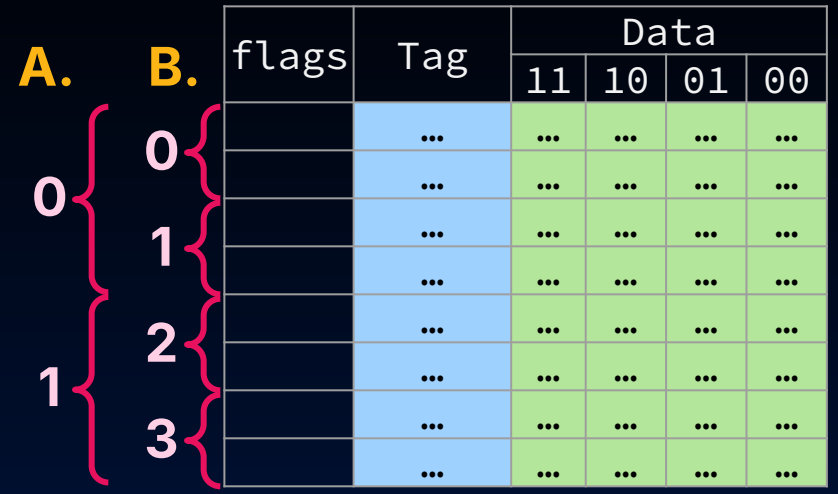
2-Way Set Associative, cache size: 16B, line size 4B

Designing Set-Associative (SA) Caches

N-Way Set Associative: groups of N cache lines are assigned a unique index.

1. Which mapping, **A or B**, represents a 32B 2-way SA Cache with 4B lines?

2. **True/False**: In set-associative caches, the valid bit should indicate whether a set contains a valid tag.

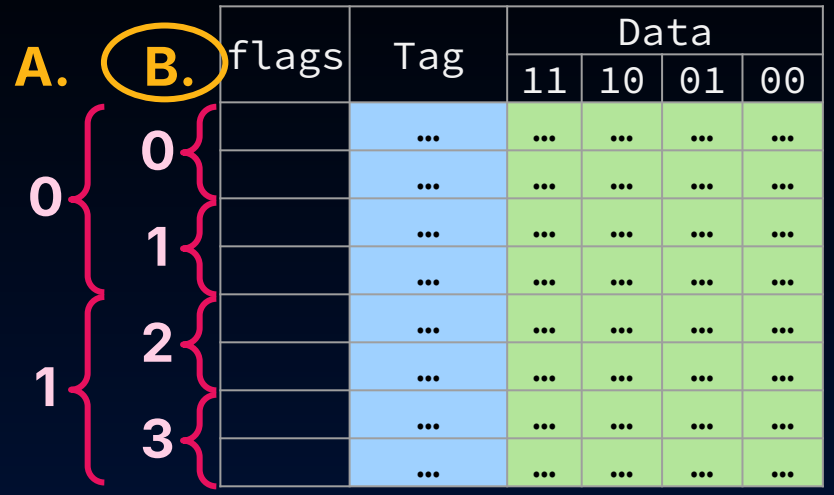


Yan, SP26

Designing Set-Associative (SA) Caches

N-Way Set Associative: groups of N cache lines are assigned a unique index.

1. Which mapping, **A or B**, represents a 32B 2-way SA Cache with 4B lines?



2. **True/False**: In set-associative caches, the valid bit should indicate whether a set contains a valid tag.

- All caches need to "warm up"
- Valid bits determine whether a **tag** is valid.
- **False**. Need to have one valid bit per block in the set!!!

2-Way 16B Set-Associative, 4B line size, write-back

- Suppose the cache starts **cold**.

1. Load byte @ `0xFE2`
   `0xFE,0b00,0b10`
2. Store byte @ `0x61C`
   `0x61,0b11,0b00`
3. Load byte @ `0x61B`
   `0x61,0b10,0b11`
4. Load byte @ `0xCAD`
   `0xCA,0b11,0b01`

	Valid	Dirty	Tag	Data 11	Data 10	Data 01	Data 00
0			…	…	…	…	…
			…	…	…	…	…
1			…	…	…	…	…
			…	…	…	…	…
2			…	…	…	…	…
			…	…	…	…	…
3			…	…	…	…	…
			…	…	…	…	…

memory address

11 … 4	3 … 2	1 0
tag	index	offset

Walkthrough

Agenda

Cache Associativity

- Set-Associative Caches
- Cache Associativity
- Types of Misses
- Cache Optimizations

From byte-addressable memory to lines/blocks

Memory, in line numbers

memory byte address

line number **offset**

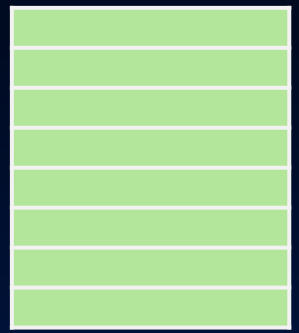
Memory, in bytes

Cache Placement Policies: Summary (1/3)

Memory, in line numbers

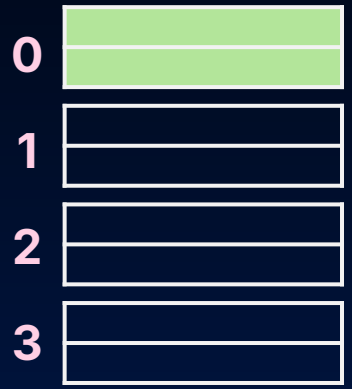
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31

Cache



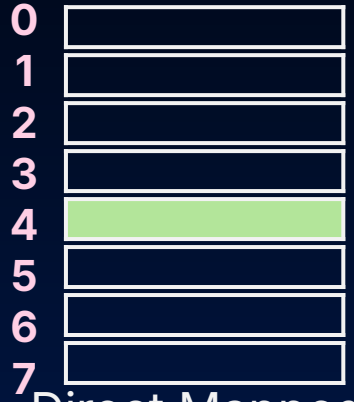
Fully Associative Cache

anywhere



2-way Set Associative Cache

anywhere in set 0 (12 mod 4)

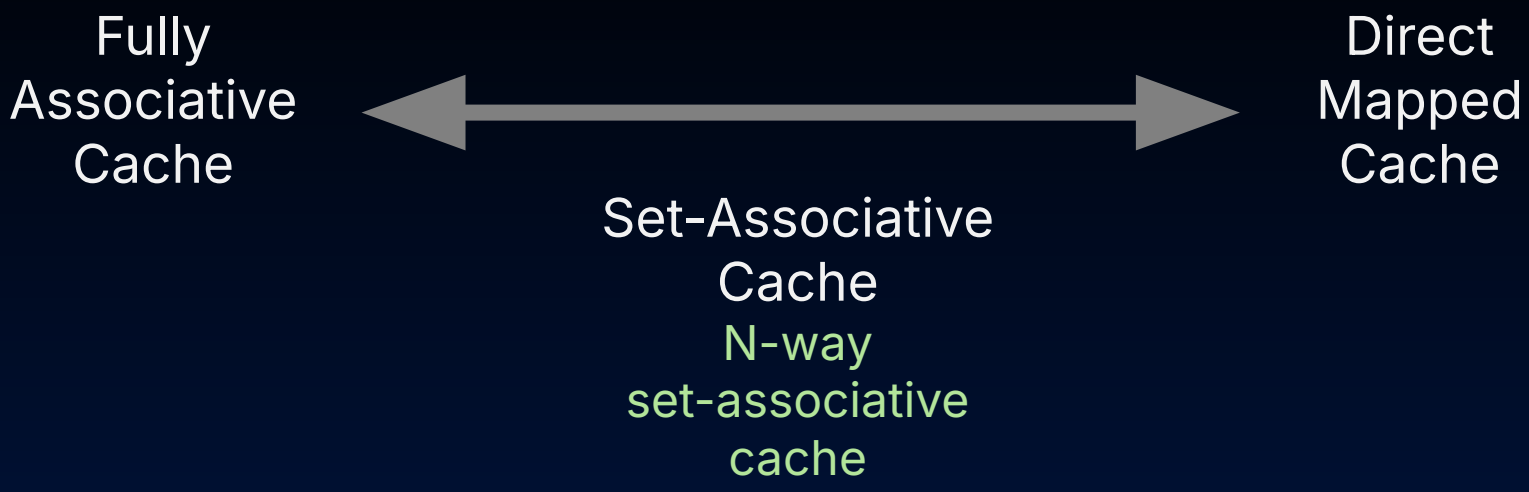


Direct Mapped Cache

Only in line 4 (12 mod 8)

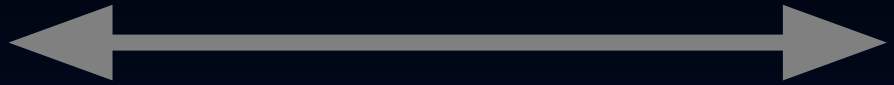
Line 12 can be placed...

Cache Placement Policies: Associativity (3/3)



Yan, SP26

Cache Placement Policies: Associativity (3/3)



Fully Associative Cache

Direct Mapped Cache

Set-Associative Cache
N-way set-associative cache

Can be considered "**<# lines>-way** set-associative cache"

Can be considered "**1-way** set-associative cache"

4-way, 8-way set-associative mostly balances the **good parts of FA** (i.e., better performance by reducing conflict misses) with **good parts of DM** (simpler hardware)

Cache design policies (2/2)

	Fully Associative Cache	Direct-Mapped Cache	N-way Set-Associative Cache
Placement policy	Data at any memory address can be associated with any cache line	Each memory address is associated with exactly one possible line in the cache.	The data can only be stored at one index (N ways/blocks per set).
Replacement policy	Must decide. LRU or FIFO or Random	Know exactly which line to replace–the one at our target index	Must decide, replace within set. LRU or FIFO or Random
Write policy	Write-back (dirty bit) or write-through	Write-back (dirty bit) or write-through	Write-back (dirty bit) or write-through
Hardware tradeoffs?	Expensive. Check all tags.	Simple. Check one tag.	Balanced ("goldilocks")

Agenda

Types of Misses

- Set-Associative Caches
- Cache Associativity
- Types of Misses
- Cache Optimizations

How to choose cache design?

Average Access Memory Time

$$\text{AMAT} = \textit{Hit Time} + \textit{Miss Rate} \times \textit{Miss Penalty}$$

We have seen **associativity** plays a big role. What types of misses are there?

Types of Misses

- **Compulsory Miss**
  - Caused by the first access to data that has never been in the cache.
- **Capacity Miss**
  - Caused when the cache cannot contain all the lines needed during the execution of a program.
  - Occur when lines were in the cache, replaced, and later retrieved.
- **Conflict Miss**
  - Multiple lines compete for the same location in the cache, even when the cache has not reached full capacity.

"Non-compulsory" miss

In this class, we will only ask you to distinguish between **compulsory and non-compulsory misses** if *given a specific memory access sequence*.

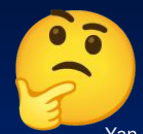
Know the definitions of all three misses!!!

Types of Misses

- **Compulsory Miss**
  - Caused by the first access to data that has never been in the cache.
- **Capacity Miss**
  - Caused when the cache cannot contain all the lines needed during the execution of a program.
  - Occur when lines were in the cache, replaced, and later retrieved.
- **Conflict Miss**
  - Multiple lines compete for the same location in the cache, even when the cache has not reached full capacity.

Which types of misses can occur for Fully Associative caches (FA)? For Direct Mapped caches (DM)? Select all that apply.

A. FA: Compulsory
B. FA: Capacity
C. FA: Conflict
D. DM: Compulsory
E. DM: Capacity
F. DM: Conflict
G. None of the above



Yan, SP26

L17: Which types of misses can occur for Fully Associative caches (FA)? For Direct Mapped caches (DM)? Select all that apply.

Compulsory Miss
- Caused by the first access to a block that has never been in the cache

Capacity Miss
- Caused when the cache cannot contain all the blocks needed during the execution of a program
- Occur when blocks were in the cache, replaced, and later retrieved

Conflict Miss
- Multiple blocks compete for the same location in the cache, even when the cache has not reached full capacity.

A. FA: Compulsory 0%

B. DM: Compulsory 0%

C. FA: Capacity 0%

D. DM: Capacity 0%

E. FA: Conflict 0%

F. DM: Conflict 0%

G. None of the above 0%

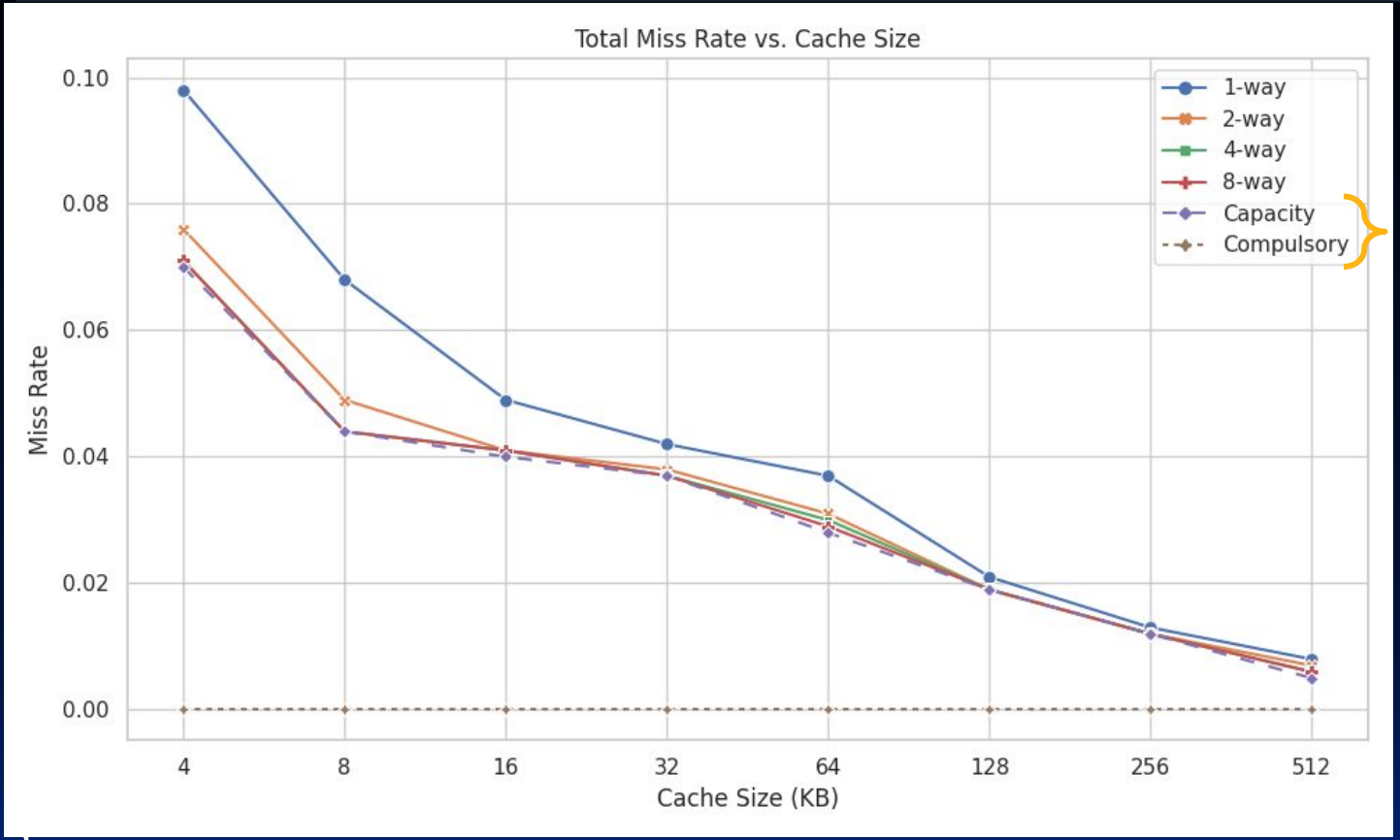
Types of Misses

- **Compulsory Miss**
  - Caused by the first access to data that has never been in the cache.
- **Capacity Miss**
  - Caused when the cache cannot contain all the lines needed during the execution of a program.
  - Occur when lines were in the cache, replaced, and later retrieved.
- **Conflict Miss**
  - Multiple lines compete for the same location in the cache, even when the cache has not reached full capacity.

Which types of misses can occur for Fully Associative caches (FA)? For Direct Mapped caches (DM)? Select all that apply.

A. FA: Compulsory
B. FA: Capacity
~~C. FA: Conflict~~
D. DM: Compulsory
E. DM: Capacity
F. DM: Conflict
G. None of the above

Associativity: Impact on AMAT



Fully associative miss rate

Agenda

Cache Optimizations

- Set-Associative Caches
- Cache Associativity
- Types of Misses
- Cache Optimizations

Impact on AMAT

Average Access Memory Time

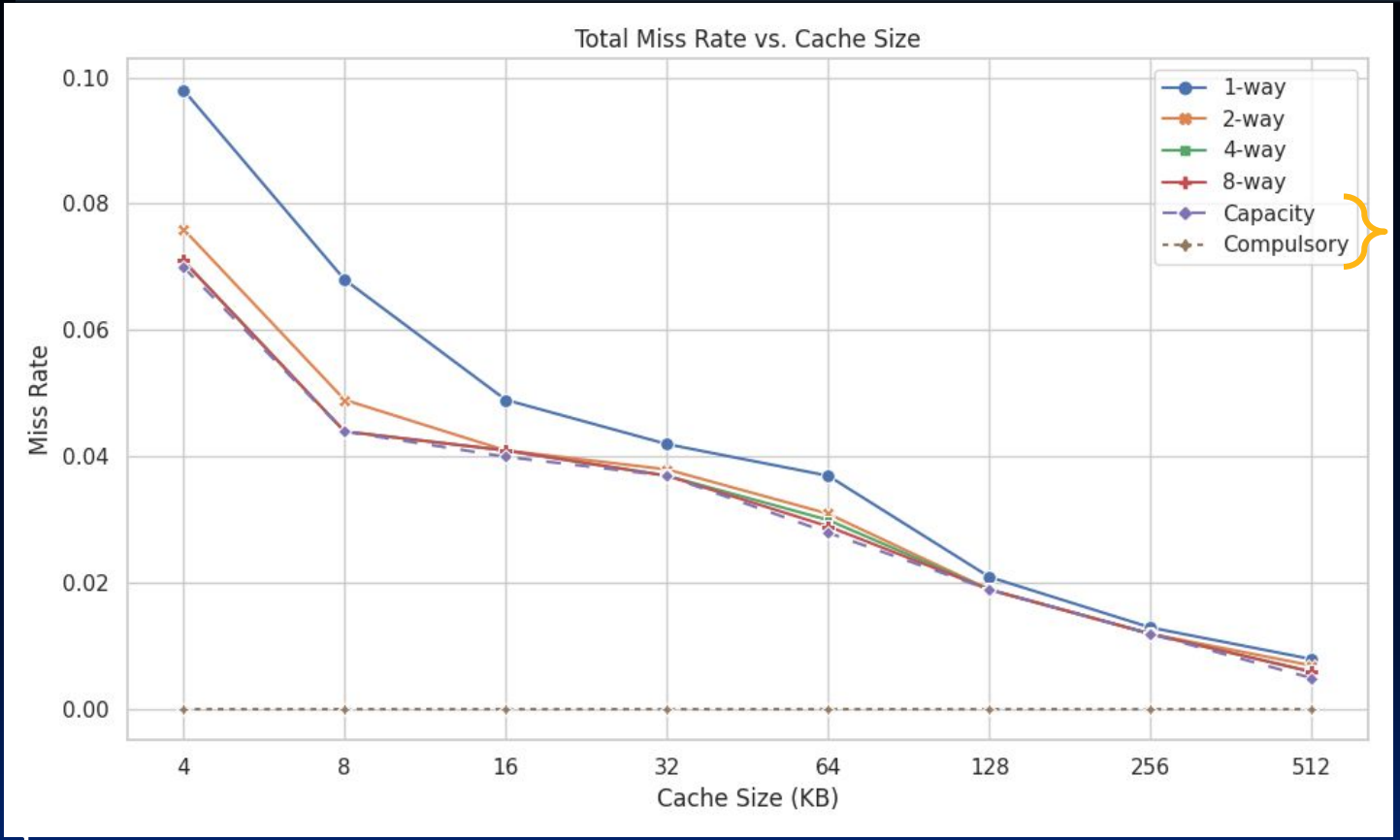
$$AMAT = Hit\ Time + Miss\ Rate \times Miss\ Penalty$$

Technique	Hit Time	Miss Penalty	Miss rate
Larger block size		–	+
Larger cache size	–		+
Higher associativity	–		+
Multilevel caches		+	

(+): good, reduces time/penalty
(–): bad, increases time/penalty

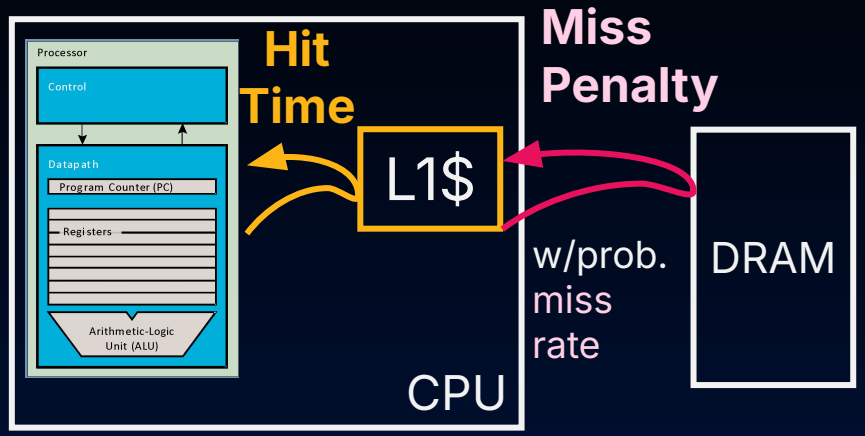
*Compare different cache designs on benchmarks

Larger Cache Size (the same graph)

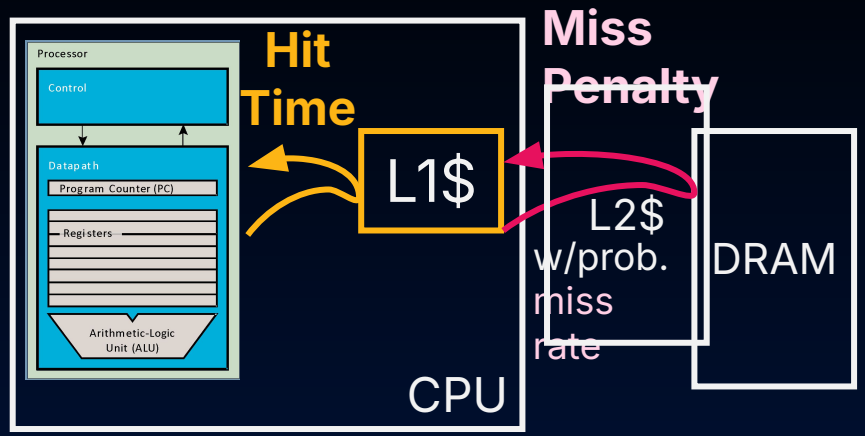


Fully associative miss rate

Multilevel caches



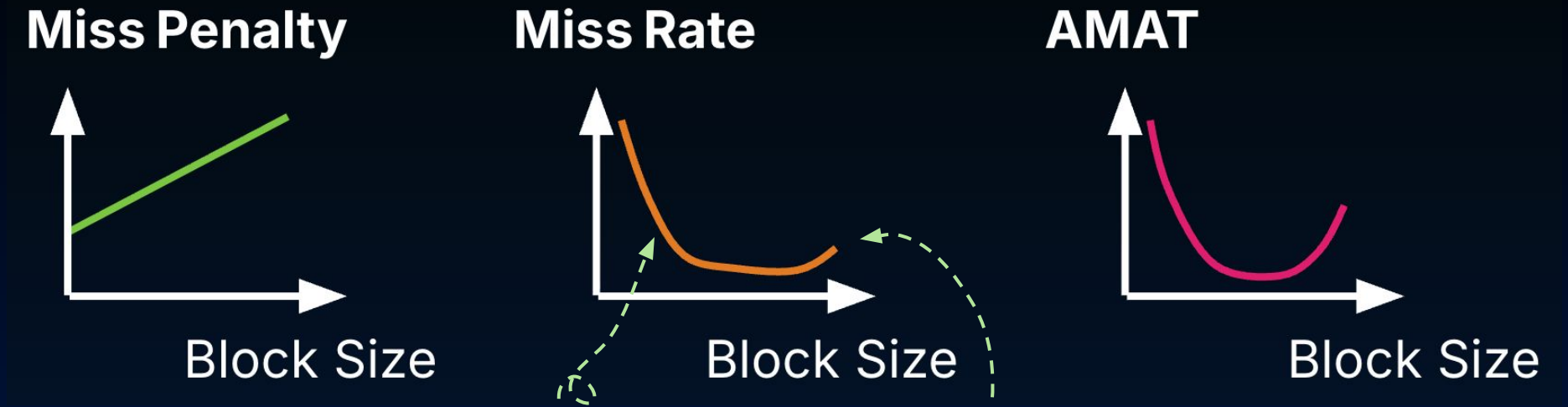
AMAT = 11 cycles



AMAT = 2.75 cycles

From Caches II: multilevel reduces AMAT because it reduces miss penalty

Larger Block Size?

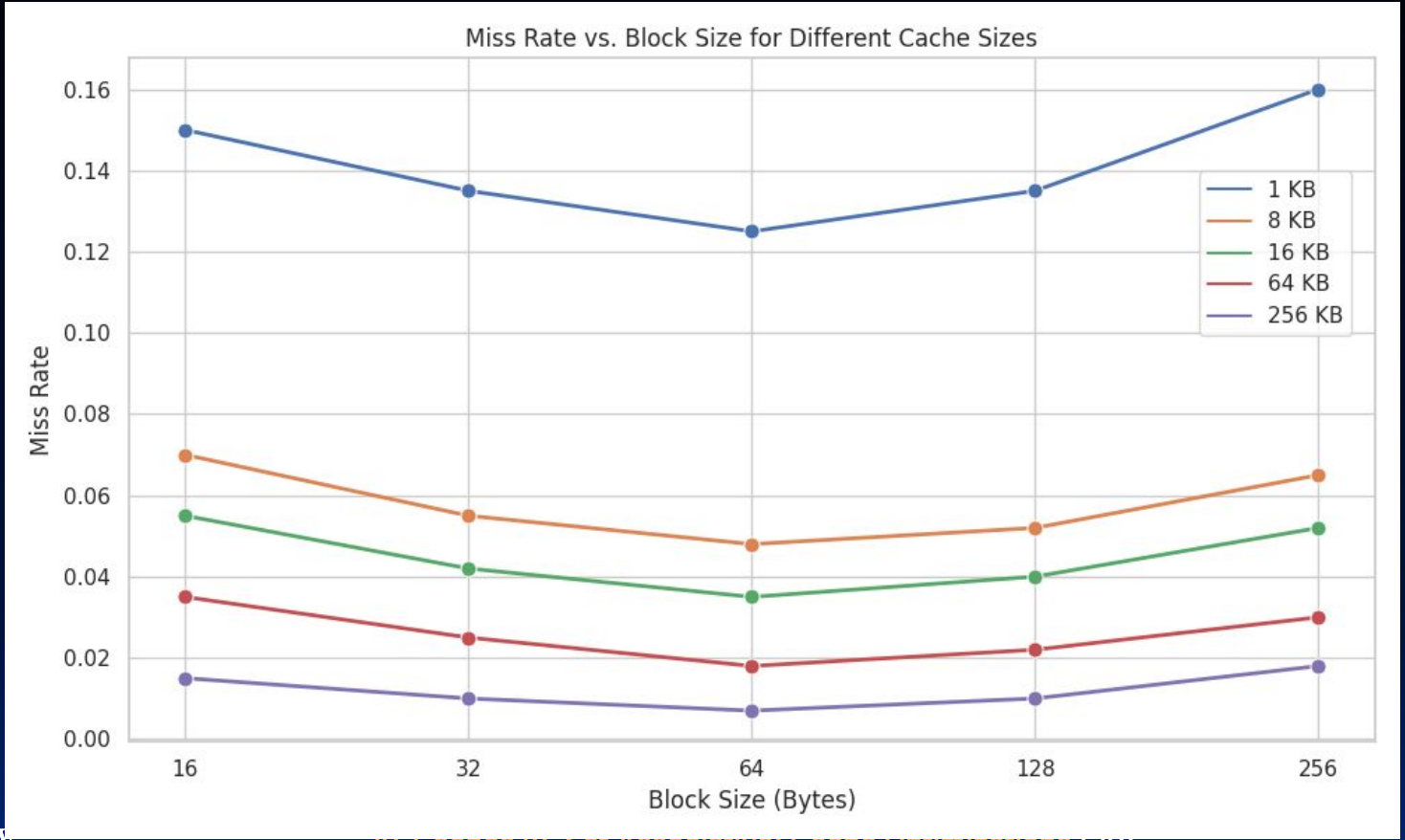


More bytes to transfer from memory

Spatial locality

Compromise temporal locality

Larger Block Size, Measured



Apple M1 Chip

	Level	Ways	Sets	Line Size	Total Size
p-core	L1I	6	512	64 B	192 KB
	L1D	8	256	64 B	128 KB
	L2	12	8192	128 B	12 MB
e-core	L1I	8	256	64 B	128 KB
	L1D	8	128	64 B	64 KB
	L2	16	2048	128 B	4 MB

Table 2: Cache configurations on M1 obtained via reading system registers.

Ravichandran and Na, et al. 2022. USENIX